

Unidade 2 — Técnicas de Prompt

Aplicações de Modelos de Linguagem Natural

- 1 Contexto
- 2 Chain-of-Thought (CoT) Prompting
- 3 ReAct Prompting

- 1 Contexto
- 2 Chain-of-Thought (CoT) Prompting
- 3 ReAct Prompting

Da Unidade 2 até aqui

- Nas seções anteriores desta unidade, vimos as duas técnicas fundamentais de prompting, que diferem principalmente pela quantidade de exemplos fornecidos ao modelo: o **zero-shot prompting** (nenhum exemplo) e o **few-shot prompting** (alguns exemplos, chamados de demonstrações).
- Agora avançamos para duas técnicas mais avançadas, voltadas especificamente para melhorar o **raciocínio** do modelo: o **Chain-of-Thought (CoT) prompting**, que estrutura o raciocínio em passos, e o **ReAct prompting**, que combina raciocínio com o uso de ferramentas e ações externas.
- Assim como nas técnicas anteriores, veremos para cada uma delas o que ela é, como funciona, exemplos práticos de aplicação, e suas principais vantagens e limitações.

Sumário

- 1 Contexto
- 2 Chain-of-Thought (CoT) Prompting
- 3 ReAct Prompting

O que é Chain-of-Thought (CoT) prompting

- **Chain-of-Thought (CoT) prompting**, ou “encadeamento de pensamento”, é uma técnica que melhora a capacidade de raciocínio dos modelos de linguagem ao dividir problemas complexos em passos organizados e sequenciais, em vez de exigir uma resposta direta e imediata.
- Essa abordagem é especialmente útil em tarefas que exigem raciocínio em múltiplas etapas, como problemas matemáticos, análise lógica e raciocínio de senso comum.
- Torna as respostas mais interpretáveis e transparentes, já que o processo de raciocínio do modelo fica explícito, e não escondido por trás da resposta final.

- Ao contrário do prompting padrão, que busca uma resposta imediata, o CoT prompting orienta o modelo a explicitar seu processo de raciocínio em etapas antes de chegar a uma conclusão.
- Esse método traz três benefícios principais:
 - a **decomposição de tarefas complexas**, em que o modelo divide sistematicamente um problema em passos menores e mais gerenciáveis;
 - a **redução de erros**, já que seguir um caminho de raciocínio estruturado diminui a chance de erros causados por “adivinhação” ou má interpretação do problema;
 - a **melhora da interpretabilidade**, uma vez que, com os passos de raciocínio explícitos, a resposta final se torna mais compreensível e mais fácil de verificar.

Exemplo 9 — Prompt sem Chain-of-Thought

- Os três exemplos a seguir, extraídos do livro *Prompt Engineering*, de Lee Boonstra (Google, 2025), mostram a evolução de uma mesma pergunta matemática: primeiro sem nenhuma instrução de raciocínio, depois com Chain-of-Thought no formato zero-shot e, por fim, com Chain-of-Thought combinado a um exemplo (one-shot).

Prompt

“Quando eu tinha 3 anos, meu(minha) parceiro(a) tinha 3 vezes a minha idade. Agora, tenho 20 anos. Qual é a idade do(a) meu(minha) parceiro(a)?”

Saída

“63 anos”

Sem nenhuma instrução para raciocinar passo a passo, o modelo erra o cálculo e responde “63 anos” — a idade correta é 26 anos.

Exemplo 10 — Chain-of-Thought no formato zero-shot

Prompt

“Quando eu tinha 3 anos, meu(minha) parceiro(a) tinha 3 vezes a minha idade. Agora, tenho 20 anos. Qual é a idade do(a) meu(minha) parceiro(a)? **Vamos pensar passo a passo.**”

Saída

- 1 Vamos assumir que a minha idade atual é “x” anos.
- 2 Quando eu tinha 3 anos, a minha idade era 3 anos.
- 3 A idade do(a) meu(minha) parceiro(a) naquela época era 3 vezes a minha idade, ou seja, $3 \times 3 = 9$ anos.
- 4 Agora tenho 20 anos, o que significa que a minha idade aumentou $20 - 3 = 17$ anos desde quando eu tinha 3 anos.
- 5 Como a idade do(a) meu(minha) parceiro(a) era 9 anos quando eu tinha 3 anos, e a minha idade aumentou 17 anos, a idade dele(a) também deve ter aumentado 17 anos.
- 6 Portanto, a idade atual do(a) meu(minha) parceiro(a) é $9 + 17 = 26$ anos. Logo, meu(minha) parceiro(a) tem 26 anos.

Ao acrescentar ao mesmo prompt a instrução “vamos pensar passo a passo”, o modelo explicita cada etapa do raciocínio e chega à resposta correta, 26 anos.

Exemplo 11 — Chain-of-Thought com um exemplo (one-shot)

Prompt — exemplo já resolvido

P: “Quando meu irmão tinha 2 anos, eu tinha o dobro da idade dele. Agora tenho 40 anos. Qual é a idade do meu irmão? Vamos pensar passo a passo.”

R: “Quando meu irmão tinha 2 anos, eu tinha $2 \times 2 = 4$ anos. Essa é uma diferença de idade de 2 anos, e eu sou mais velho(a). Agora tenho 40 anos, então meu irmão tem $40 - 2 = 38$ anos. A resposta é 38.”

P: “Quando eu tinha 3 anos, meu(minha) parceiro(a) tinha 3 vezes a minha idade. Agora, tenho 20 anos. Qual é a idade do(a) meu(minha) parceiro(a)? Vamos pensar passo a passo.”

R:

Saída

“Quando eu tinha 3 anos, meu(minha) parceiro(a) tinha $3 \times 3 = 9$ anos. Essa é uma diferença de idade de 6 anos, e meu(minha) parceiro(a) é mais velho(a). Agora tenho 20 anos, então meu(minha) parceiro(a) tem $20 + 6 = 26$ anos. A resposta é 26.”

Ao incluir no prompt uma pergunta semelhante já resolvida passo a passo antes da nova pergunta, o modelo chega a uma resposta correta e mais direta, também 26 anos.

Vantagens do Chain-of-Thought prompting

- **Raciocínio lógico mais estruturado:** incentiva um pensamento organizado, reduzindo a chance de conclusões incorretas.
- **Maior interpretabilidade:** torna as respostas geradas pelo modelo mais fáceis de acompanhar e de verificar.
- **Melhor desempenho em tarefas complexas:** é especialmente eficaz em tarefas que exigem raciocínio profundo, como resolução de problemas matemáticos, dedução lógica e perguntas que envolvem múltiplos passos de inferência.
- **Melhora a generalização:** potencializa o desempenho do modelo tanto em contextos zero-shot quanto few-shot, facilitando o alinhamento com tarefas pouco familiares.

Limitações do Chain-of-Thought prompting

- **Maior uso de tokens:** gerar explicações passo a passo produz respostas mais longas, o que aumenta o custo computacional.
- **Dependência da qualidade dos passos:** a eficácia do CoT depende de passos de raciocínio bem estruturados; prompts de CoT mal elaborados podem levar a conclusões equivocadas.
- **Utilidade limitada em tarefas simples:** tarefas que não exigem raciocínio em múltiplas etapas podem não se beneficiar significativamente do CoT, sendo resolvidas de forma mais eficiente com respostas diretas.

- 1 Contexto
- 2 Chain-of-Thought (CoT) Prompting
- 3 ReAct Prompting

O que é ReAct prompting

- **ReAct** é a abreviação de *Reasoning and Acting* (“Raciocínio e Ação”).
- É uma técnica de prompting que combina, dentro do próprio processo de geração da resposta do modelo, etapas de raciocínio — como as vistas no Chain-of-Thought — com a tomada de ações, isto é, a interação com ferramentas ou ambientes externos, incorporando o resultado dessas ações de volta ao raciocínio do modelo.
- Essa abordagem interativa permite que o modelo resolva problemas mais complexos, busque informações adicionais e adapte seu raciocínio com base em feedback obtido em tempo real, o que costuma resultar em soluções mais precisas e robustas em tarefas que exigem interação dinâmica e aquisição de conhecimento.

Como funciona — o ciclo do ReAct

- O ReAct prompting funciona por meio de um ciclo com quatro etapas principais:
 - ① **Raciocínio:** o modelo gera passos de análise sobre o problema e formula um plano de ação.
 - ② **Ação:** o modelo age de acordo com esse raciocínio — por exemplo, consultando uma base de conhecimento, fazendo uma busca em um mecanismo de pesquisa ou utilizando alguma outra ferramenta externa.
 - ③ **Observação:** o modelo observa o resultado ou o retorno obtido a partir dessa ação.
 - ④ **Atualização do raciocínio:** o modelo revisa seu raciocínio com base nas observações feitas, podendo ajustar seu plano ou realizar novas ações até resolver o problema.

Figura 1 — Ciclo do ReAct prompting

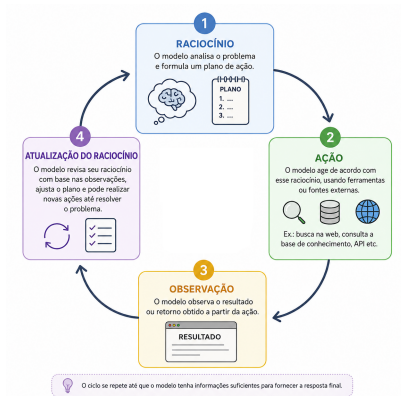


Figura 1: Ciclo do ReAct prompting.

Exemplo 12 — Pergunta em múltiplos saltos (HotpotQA)

- O exemplo a seguir é baseado no **HotpotQA**, um conjunto de dados de perguntas e respostas que exige raciocínio em múltiplos saltos — ou seja, a resposta não está disponível em uma única fonte, sendo necessário combinar mais de uma informação para chegar a ela.

Prompt enviado ao modelo

“Resolva a pergunta a seguir alternando entre Raciocínio, Ação e Observação, até chegar a uma resposta final.
Pergunta: Qual hotel sedia o espetáculo ‘Mystère’, do Cirque du Soleil, e quantos quartos ele tem?”

Exemplo 12 — Passo a passo do agente

Passo a passo do agente e resposta final

Raciocínio 1: Preciso primeiro descobrir qual hotel sedia o espetáculo “Mystère”, do Cirque du Soleil, e depois descobrir o número de quartos desse hotel.

Ação 1: Pesquisar por “Cirque du Soleil Mystère show”.

Observação 1: O espetáculo “Mystère” é sediado no Treasure Island Hotel and Casino, em Las Vegas.

Raciocínio 2: Agora que sei qual é o hotel, preciso descobrir quantos quartos o Treasure Island Hotel and Casino tem.

Ação 2: Pesquisar por “Treasure Island Hotel and Casino number of rooms”.

Observação 2: O Treasure Island Hotel and Casino tem 2.884 quartos.

Raciocínio 3: Portanto, o Treasure Island Hotel and Casino sedia o espetáculo “Mystère” e tem 2.884 quartos.

Ação 3 (final): Finalizar: o Treasure Island Hotel and Casino sedia o espetáculo do Cirque du Soleil “Mystère” e tem 2.884 quartos.

O modelo alterna entre raciocinar sobre o que ainda precisa descobrir e agir para buscar essa informação, usando cada observação para planejar o próximo passo, até reunir todos os dados necessários para responder à pergunta original.

Vantagens do ReAct prompting

- **Resolução dinâmica de problemas:** permite que o modelo lide com tarefas que exigem interação e adaptação contínuas.
- **Aquisição de conhecimento:** possibilita obter e utilizar informações em tempo real a partir de fontes externas.
- **Maior precisão:** incorpora feedback e observações reais ao processo de geração da resposta, tornando-a mais precisa.
- **Explicabilidade:** revela tanto o raciocínio quanto as ações do modelo, tornando seu processo de decisão mais transparente.

- **Complexidade de implementação:** colocar o ReAct em prática exige integração com ferramentas e ambientes externos, o que aumenta a complexidade do sistema.
- **Custo computacional:** executar ações e observar seus resultados pode ser custoso e aumentar o tempo de resposta.
- **Preocupações de segurança:** permitir que o modelo interaja com ambientes externos cria o risco de ações não intencionais ou potencialmente prejudiciais.

- O **Chain-of-Thought prompting** melhora o raciocínio do modelo ao organizá-lo em passos explícitos e sequenciais, tornando a resposta mais interpretável, mas todo o processo permanece interno ao modelo, sem contato com o mundo exterior.
- O **ReAct prompting** estende essa ideia: além de raciocinar em etapas, o modelo intercala esse raciocínio com **ações** sobre ferramentas ou ambientes externos e com a **observação** dos resultados dessas ações, atualizando seu raciocínio a cada novo dado obtido.
- Assim, o ReAct é particularmente útil quando a resposta depende de informações que o modelo não possui de antemão e que precisam ser buscadas e combinadas ao longo do processo — como no exemplo de perguntas em múltiplos saltos do HotpotQA.